Escuela de Doctorado
y Estudios de Posgrado
Universidad de La Laguna

PROYECTO DIGITAL TWIN

Informática Industrial Avanzada

Jolehisy Acevedo Medina

18/03/2026

Máster en Ingeniería Industrial

Universidad de La Laguna

ÍNDICE

1. Introducción y Objetivos 3

2. Selección y Adaptación del Modelo Digital 4

2.1. Sintonización del Controlador PID Discreto 4

2.2. Modelado del Entorno Real (Fidelidad del Gemelo Digital) 6

2.2.1. Dinámica del Actuador y Respuesta Térmica 6

2.2.2. Turbulencia Atmosférica Estocástica (Mach Variable) 6

2.2.3. Estabilidad en Atmósfera No Perturbada (Altitud Dinámica) 6

2.2.4. Digitalización y Ruido de Medida (ADC/Sensores) 7

3. Arquitectura del Sistema: Subsistemas y Enmascaramiento 7

3.1. Segmentación de Componentes 7

3.2. Diseño de la Máscara de Control 8

3.3. Interfaz de Señales 8

4. Generación de Código y Validación (Software-in-the-Loop) 9

4.1. Procedimiento de Configuración y Compilación 9

4.2. Procesamiento y Ejecución del Código (Software-in-the-Loop) 10

4.2.1. Configuración del Entorno de Compilación 10

4.2.2. Proceso de Compilación y Enlace 11

4.2.3. Ejecución y Verificación de Resultados 11

5. Empaquetado del Modelo Protegido (.slxp) 12

5.1. Pasos realizados 12

6. Conclusión General del Proyecto 13

1. Introducción y Objetivos

El presente proyecto tiene como objetivo el diseño, implementación y validación de un sistema de control digital de empuje (FADEC) para un motor de aviación mediante un modelo de turbina de gas comercial (*Turbofan Engine System*). El trabajo se ha estructurado para dar respuesta a los requisitos técnicos del programa, evolucionando un modelo termodinámico hacia un "Digital Twin" de alta fidelidad.

Para cumplir con las directrices del enunciado, el desarrollo se divide en los siguientes bloques:

- **Selección y uso de un modelo digital:** Se ha optado por un modelo de alta fidelidad de una turbina de gas, implementando un actuador de combustible inteligente (*Throttle*) gestionado por un controlador PID discreto para mantener el empuje constante.
- **Realismo mediante sensores y perturbaciones:** Modificación del modelo base para incluir condiciones reales de operación aeroespacial, tales como el muestreo de datos del sensor mediante un retenedor de orden cero (ZOH) a 100 Hz, la saturación física de la válvula, y un escenario de perturbaciones aerodinámicas.
- **Subsistema de control enmascarado:** Diseño de un bloque de control FADEC encapsulado y parametrizado mediante una máscara de Simulink, permitiendo una configuración ágil de las ganancias del PID digital y los tiempos de muestreo sin acceder a la lógica interna.
- **Generación de código C++ y validación:** Transformación del algoritmo de control discreto a código C++ integrado, verificando mediante el *Simulation Data Inspector* que la respuesta de la señal de control generada es idéntica a la simulación original.
- **Protección del modelo (Digital Twin):** Configuración del sistema completo (Controlador + Turbina) para su distribución en un formato protegido (.slxp), permitiendo realizar pruebas de rendimiento ante cambios de referencias de vuelo sin exponer la propiedad intelectual del diseño.

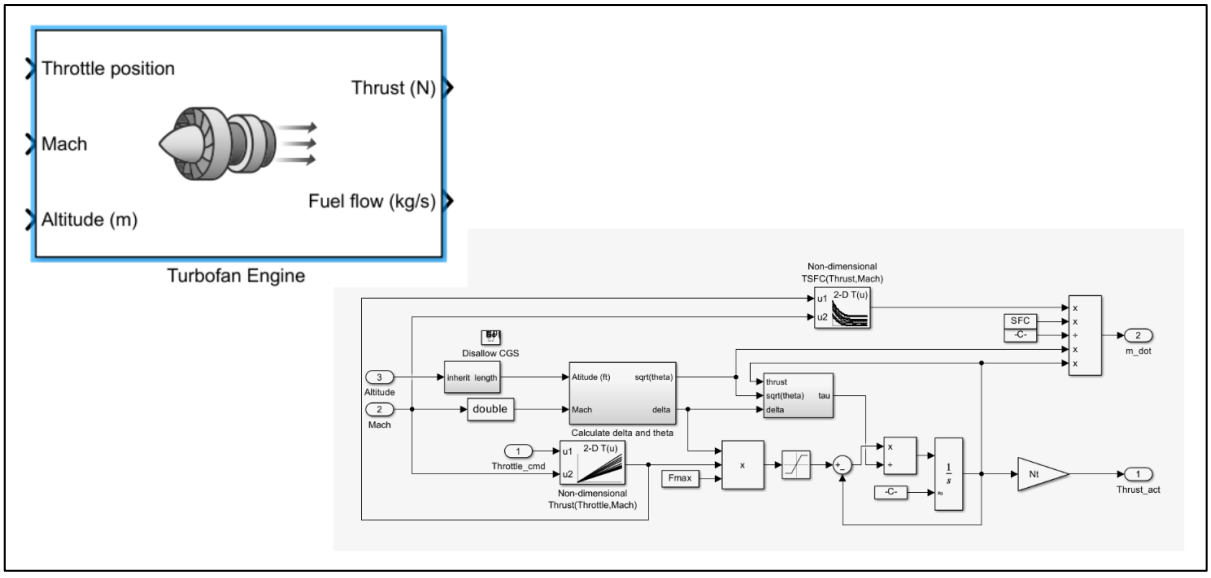

Figura 1. Modelo de Turbina elegido

2. Selección y Adaptación del Modelo Digital

Para este proyecto se ha seleccionado el modelo de referencia de la librería *Aerospace Blockset* de Simulink (Turbofan Engine System). Este bloque representa la física continua y no lineal de una turbina de gas. Sin embargo, para cumplir con los objetivos de control digital específicos y acercar la simulación a la realidad del hardware de a bordo, se han realizado las siguientes adaptaciones:

- **Discretización y Emulación del Sensor (ADC):** La turbina real opera en tiempo continuo, pero el ordenador de vuelo (FADEC) procesa la información de forma discreta. Se ha introducido un bloque **Zero-Order Hold (ZOH)** en el lazo de realimentación del empuje (*Thrust*). Este bloque actúa como el conversor analógico-digital del sistema, muestreando la señal física a una frecuencia de 100 Hz ($T_s = 0.01$ s). El controlador PID se ha sincronizado a esta misma frecuencia para evitar la pérdida de datos y crear un sistema de control *single-rate*.
- **Limitaciones Físicas del Actuador (Saturación y Anti-windup):** La variable manipulada por el controlador es la posición de la válvula de combustible (*Throttle position*). Físicamente, esta válvula tiene un recorrido limitado entre el cierre casi total y la apertura máxima. Por ello, se ha configurado la salida del PID Discreto con una saturación estricta entre 0 y 1. Adicionalmente, se ha activado el mecanismo *Anti-windup* (método *clamping*) para evitar que el término integral se acumule al infinito cuando el actuador alcanza su límite físico, garantizando la estabilidad del sistema frente a grandes errores de consigna.

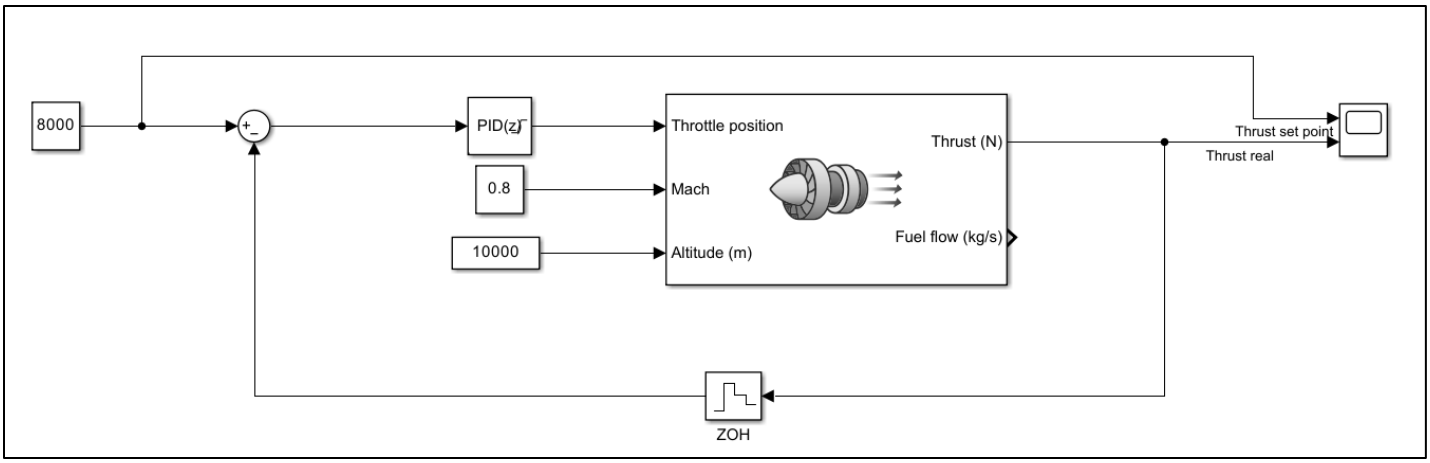

Figura 2. Modelo con controlador PID

2.1. Sintonización del Controlador PID Discreto

El ajuste de las ganancias del controlador se ha realizado priorizando la estabilidad absoluta y la integridad física de la turbina. En la industria aeroespacial, un exceso de inyección de combustible (sobreoscilación u *overshoot*) provoca picos de temperatura en los gases de escape que pueden fundir los álabes de la turbina. Por tanto, el objetivo principal de la sintonización ha sido obtener una respuesta suave, sin picos, y con un error en régimen permanente nulo.

La sintonización final responde a las siguientes consideraciones técnicas:

- **Desnivel de Magnitudes (Ganancias P e I):** Existe una enorme diferencia de escala entre la señal de error (medida en miles de Newtons) y la señal de control (válvula operando en el rango [0, 1]). Por este motivo, las ganancias requeridas son matemáticamente muy pequeñas. Se han establecido valores empíricos de **P = 0.001** e **I = 0.005**, logrando que el sistema alcance la de consigna de forma asintótica y segura.
- **Ausencia de Acción Derivativa (D = 0):** El controlador opera puramente como un regulador PI. En la práctica real, la lectura del empuje mediante sensores de presión conlleva ruido de alta frecuencia por las vibraciones del motor. Implementar una ganancia derivativa amplificaría este ruido, enviando comandos erráticos y de alta frecuencia a la válvula de combustible, lo que provocaría inestabilidad y un desgaste mecánico prematuro (fatiga) en el actuador.
- **Gestión del esfuerzo de control:** Gracias a las ganancias seleccionadas y al uso del método *Anti-windup* (Clamping), el integrador no se satura durante la fase de aceleración inicial, garantizando que la válvula se abra al máximo permitido solo durante el tiempo estrictamente necesario.

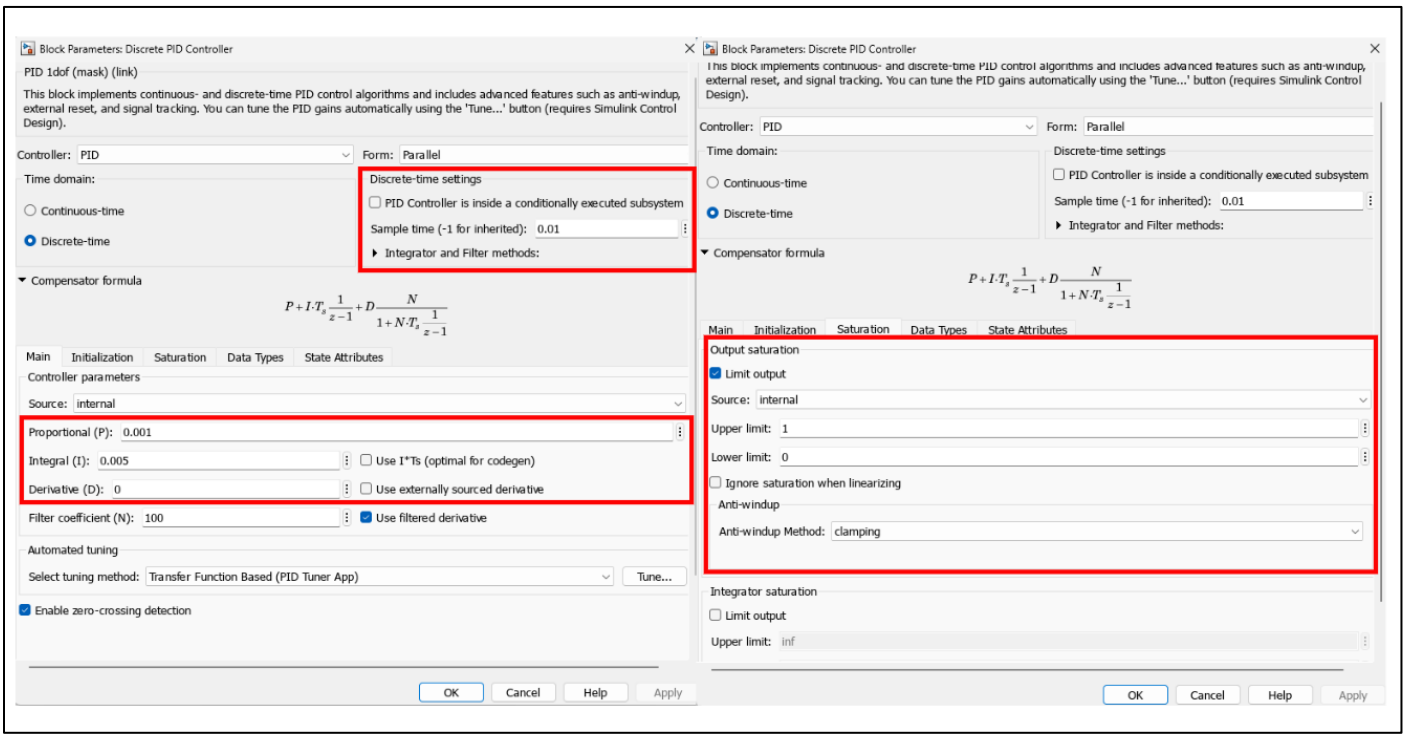

Figura 3. Sintonización y adecuación del PID

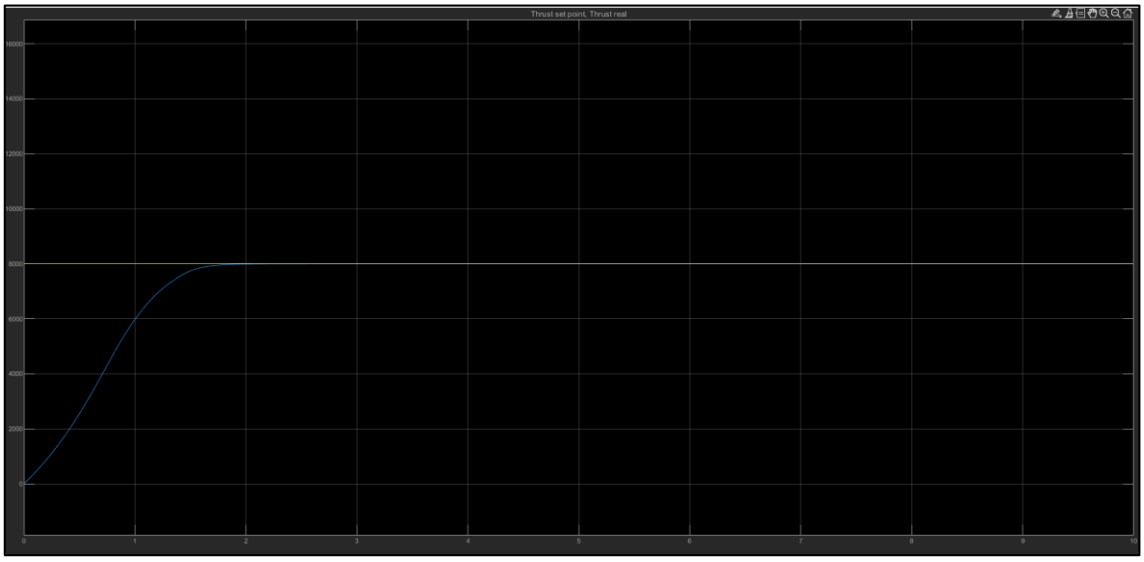

Figura 4. Respuesta del sistema con el PID sintonizado

2.2. Modelado del Entorno Real (Fidelidad del Gemelo Digital)

Para que el modelo no sea una simulación ideal, se ha dotado al sistema de una arquitectura de alta fidelidad que reproduce las condiciones físicas y ambientales de un vuelo real. Este apartado detalla los elementos de realismo inyectados en el modelo.

2.2.1. Dinámica del Actuador y Respuesta Térmica

Se ha integrado una función de transferencia de primer orden ($\frac{1}{0.5s+1}$) entre el controlador FADEC y la turbina. Esto modela la inercia mecánica de la válvula de combustible y el retardo térmico de la cámara de combustión.

- **Propósito:** Evitar respuestas instantáneas físicamente imposibles y obligar al PID a gestionar un sistema con retardo.

2.2.2. Turbulencia Atmosférica Estocástica (Mach Variable)

En lugar de un Número de Mach estático, el motor se ha expuesto a un flujo de aire variable. Utilizando un generador de ruido aleatorio filtrado, se simula la turbulencia de pequeña escala.

- **Efecto:** El sistema experimenta variaciones continuas en la presión dinámica de entrada, obligando al PID a realizar micro-ajustes permanentes en el *throttle* para estabilizar el empuje.

2.2.3. Estabilidad en Atmósfera No Perturbada (Altitud Dinámica)

Se ha modelado la altitud combinando una base estática (10.000 m) con una componente senoidal de baja frecuencia. Esto representa las corrientes térmicas y ondas de montaña que provocan cambios sutiles en la densidad del aire.

- **Resultado:** El controlador demuestra su capacidad de seguimiento dinámico, compensando los cambios de densidad para mantener los 8.000 N constantes.

2.2.4. Digitalización y Ruido de Medida (ADC/Sensores)

Para cerrar el bucle, se ha simulado la cadena de adquisición de datos mediante un bloque *Zero-Order Hold* (muestreo a 0.01s) y la inyección de **ruido blanco** en la señal de empuje.

- **Validación:** Esto garantiza que el algoritmo PID es robusto frente a las vibraciones del sensor y que no amplifica el ruido de alta frecuencia hacia el actuador.

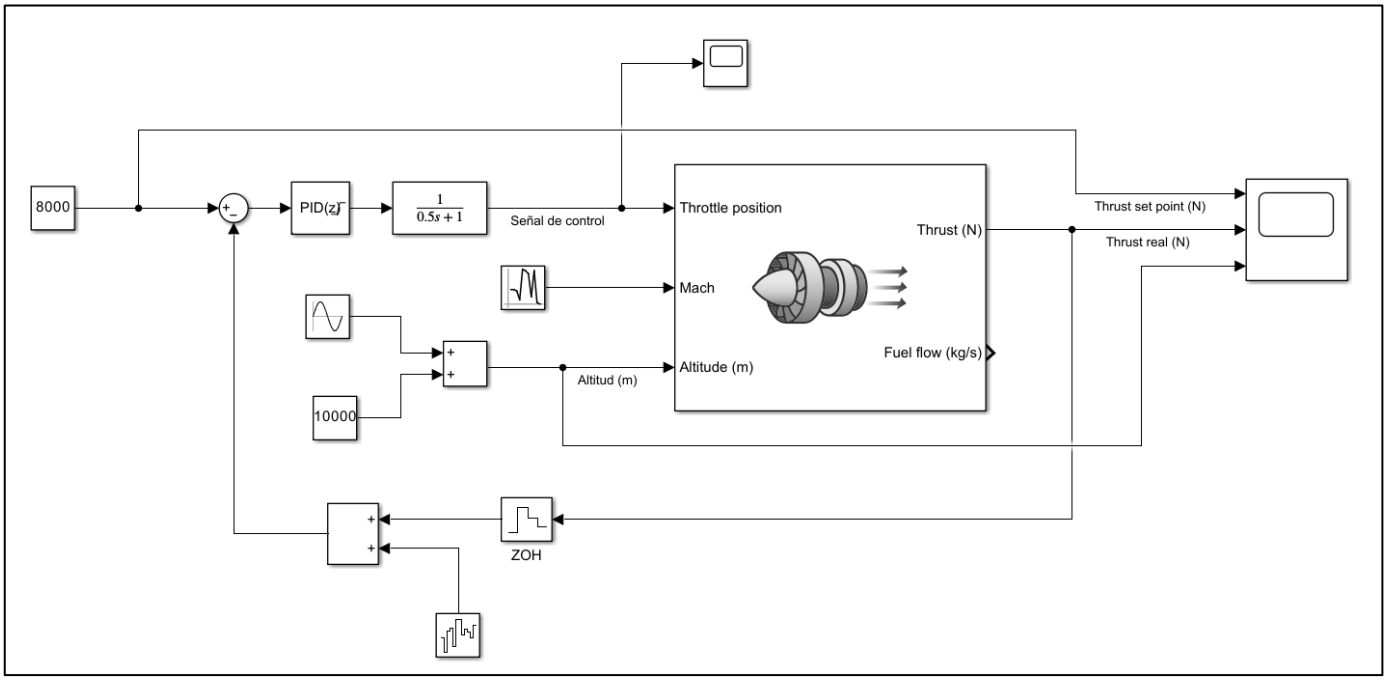

Figura 5. Planta con perturbaciones añadidas

3. Arquitectura del Sistema: Subsistemas y Enmascaramiento

Para gestionar la complejidad del Gemelo Digital y facilitar su portabilidad a sistemas reales, el modelo se ha estructurado en dos bloques funcionales independientes: la **Unidad de Control** y la **Planta Propulsiva (Turbofán)**.

3.1. Segmentación de Componentes

Se han creado dos subsistemas diferenciados para separar el software del hardware:

- **Subsistema de Control:** Encapsula el algoritmo PID discreto y la dinámica del actuador. Es el bloque que se derivará el código C++ final.
- **Subsistema de Planta y Entorno:** Este bloque agrupa el modelo termodinámico de la turbina junto con todas las fuentes de perturbación implementadas (ruido de sensores, variaciones de Mach

estocásticas y rampas de altitud). Al estar separado, permite testear el mismo controlador bajo diferentes condiciones ambientales sin modificar el algoritmo.

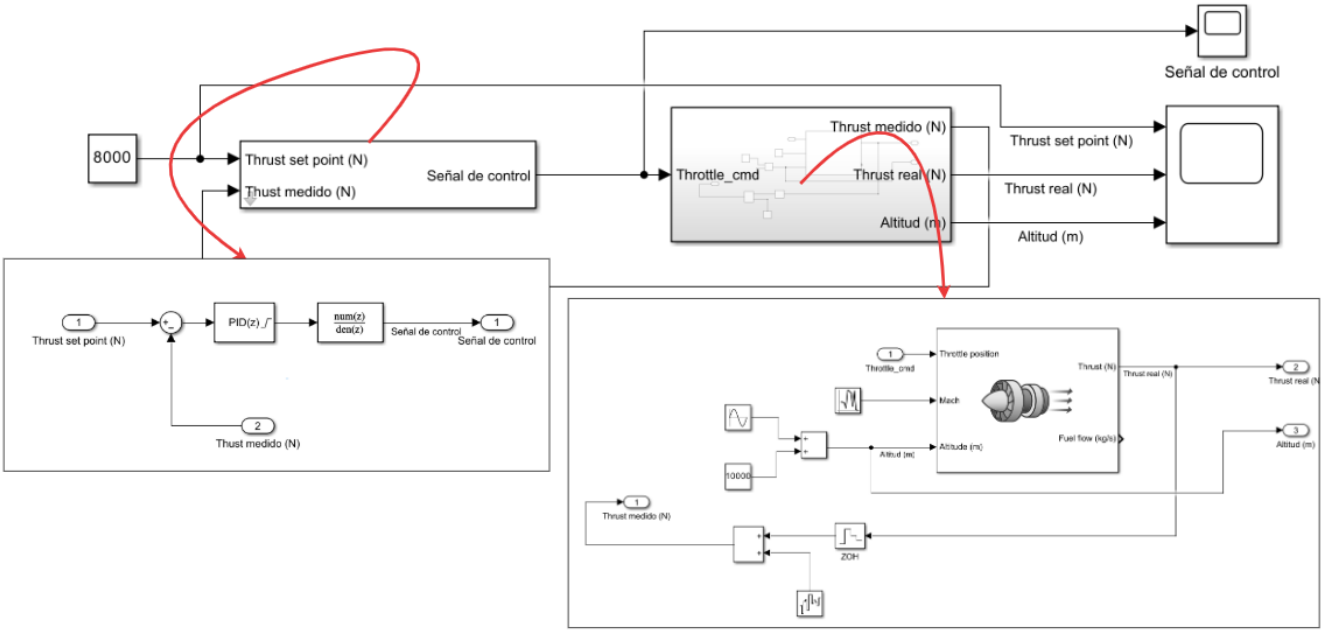

Figura 6. Planta dividida en submódulos

3.2. Diseño de la Máscara de Control

Se ha diseñado una **interfaz de usuario personalizada (Mask)** para el bloque del controlador. Esta capa de abstracción permite interactuar con el modelo de forma segura y eficiente:

- **Parametrización Dinámica:** A través del menú de la máscara, se han vinculado las variables internas del bloque PID (K_p, K_i, K_d) y de la inercia del actuador (τ) con parámetros externos.
- **Ventajas:** Esto permite realizar el sintonizado de ganancias y pruebas de sensibilidad sin necesidad de acceder al esquema interno de bloques. Además, garantiza que cualquier cambio en la máscara se actualice automáticamente en todos los componentes internos vinculados, reduciendo el riesgo de errores de configuración.

3.3. Interfaz de Señales

El diseño se ha completado definiendo puertos de entrada y salida normalizados. El controlador recibe las referencias de empuje y las señales de los sensores, mientras que la planta procesa el caudal de combustible calculado y devuelve el empuje real, cerrando así el lazo de control de forma modular.

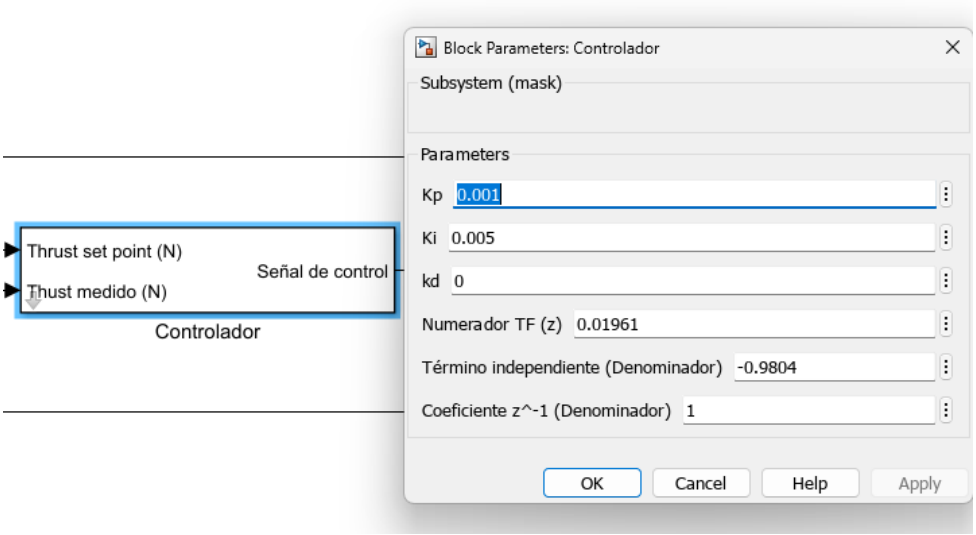

Figura 7. Máscara creada en el controlador

4. Generación de Código y Validación (Software-in-the-Loop)

Una vez validado el comportamiento del **Gemelo Digital** en el entorno de simulación, el paso final consiste en transformar el modelo gráfico del controlador en un algoritmo ejecutable en lenguaje C++. Este proceso es fundamental para garantizar que la lógica de control diseñada sea portadora a una **ECU** real o a un entorno de pruebas de alta fidelidad fuera de Simulink.

4.1. Procedimiento de Configuración y Compilación

Para realizar la transición de bloques a código fuente, se ha seguido un flujo de trabajo estándar de ingeniería rápida, asegurando que el software resultante sea determinista y eficiente:

1. **Ajuste del Solver (Tiempo Real):** En el menú de *Model Settings*, se ha configurado un esquema de **Paso Fijo (Fixed-step)** con un tiempo de muestreo de **0.01 s**. Esta configuración es crítica, ya que define la frecuencia de ejecución del controlador (100 Hz), permitiendo que el código sea compatible con sistemas de tiempo real.
2. **Selección del Target (GRT):** Se ha seleccionado el sistema de generación **Generic Real-Time (grt.tlc)**. Este perfil está optimizado para generar código C/C++ que puede compilarse como un ejecutable independiente (.exe), ideal para realizar validaciones de tipo **Software-in-the-Loop (SIL)** en el PC de desarrollo.
3. **Generación mediante "Build Subsystem":** Tras verificar la integridad de las señales, se ha ejecutado el comando de compilación sobre el subsistema del Controlador. Este proceso traduce automáticamente los bloques (PID, filtros y saturaciones) en funciones matemáticas de bajo nivel, invocando al compilador del sistema para generar los archivos fuente (.c y .h) y el binario final.

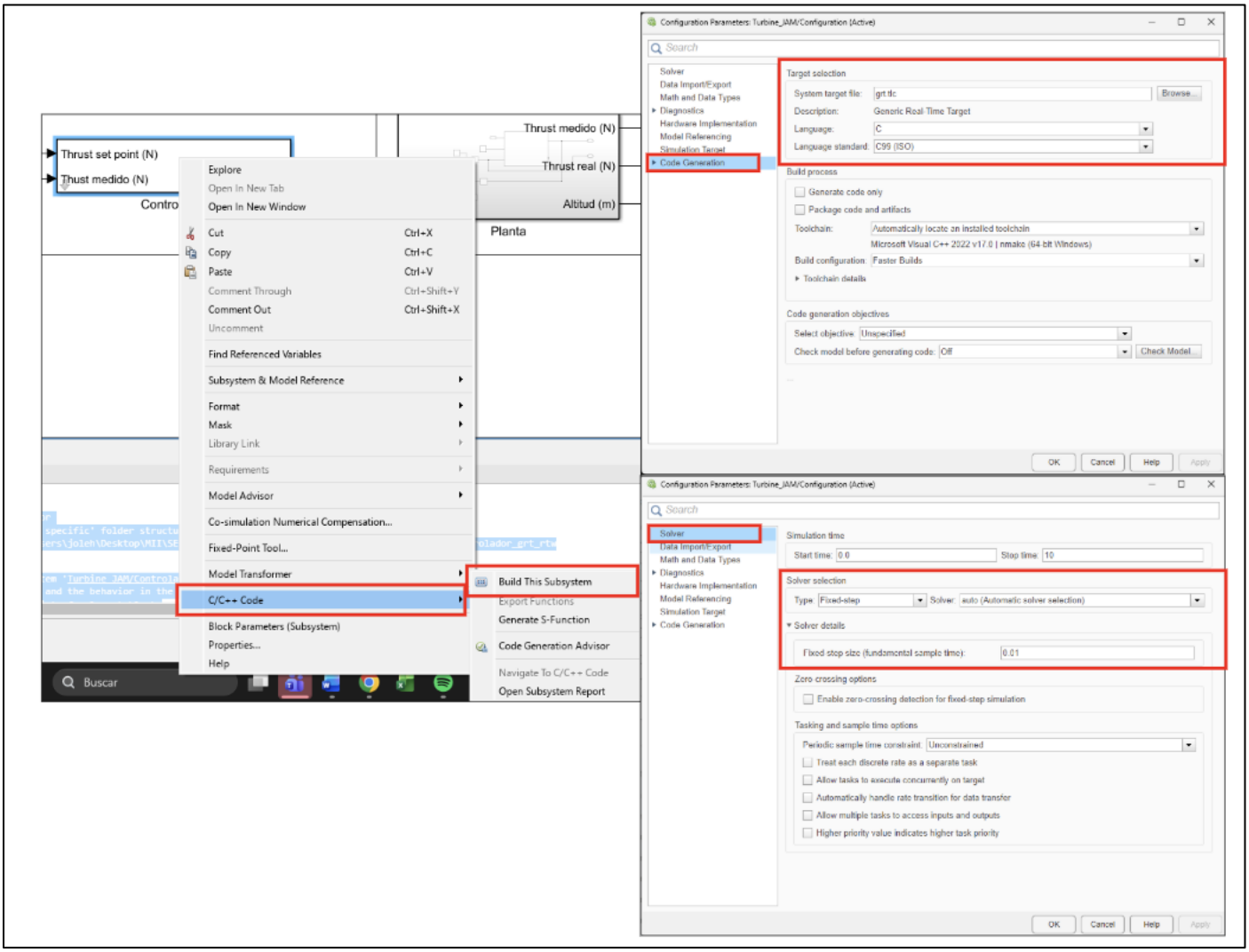

Figura 8. Procedimiento de generación del código C++

4.2. Procesamiento y Ejecución del Código (Software-in-the-Loop)

Una vez generados los archivos fuente por Simulink, el siguiente paso consiste en procesarlos fuera del entorno de MATLAB para verificar su portabilidad y funcionamiento en un sistema operativo estándar.

4.2.1. Configuración del Entorno de Compilación

Se ha utilizado el compilador **MSVC (Microsoft Visual C++)** integrado en Visual Studio Code. Para poder procesar el código, se ha creado un archivo de entrada principal (**main.cpp**) que actúa como arnés de pruebas, encargándose de:

1. **Instanciar** las estructuras de datos del controlador.
2. **Alimentar** las señales de entrada (Setpoint y Medida).
3. **Llamar** a la función de paso de cálculo (Controlador_step).

4.2.2. Proceso de Compilación y Enlace

Debido a que el código generado incluye dependencias de registro de datos (logging) propias de MATLAB, se ha implementado un archivo de soporte (**parche.c**) con funciones vacías para permitir un enlace exitoso sin necesidad de librerías externas pesadas.

La compilación se realiza de forma directa mediante la terminal:

cl /EHsc main.cpp Controlador.c Controlador_data.c rt_nonfinite.c parche.c

4.2.3. Ejecución y Verificación de Resultados

Para validar el código C++ generado, se ha realizado una comparativa técnica entre el ejecutable y el modelo de Simulink. Se ha seleccionado deliberadamente un instante de tiempo inicial ($t = 0.01$ s) para realizar esta prueba por los siguientes motivos:

1. **Aislamiento de la Acción Integral:** Al elegir un diferencial de tiempo tan pequeño tras el arranque, nos aseguramos de que el término Integral (I) del PID no haya tenido tiempo de acumular un error histórico significativo. Esto permite validar la **respuesta pura** de la lógica proporcional y la configuración de las ganancias sin que la "memoria" del controlador enmascare los resultados.
2. **Sincronización de Estados:** En este instante ($t = 0.01$ s), tanto el modelo de Simulink como el código embebido parten de condiciones iniciales idénticas (estados nulos), lo que permite una comparación directa y limpia de la aritmética del procesador.

Resultados obtenidos:

- **Simulink (Telemetría):** En $t = 0.01$ s, la señal de control muestra un valor de 0.0196.
- **Código C++ (Consola):** El ejecutable devuelve un valor de 0.01961.

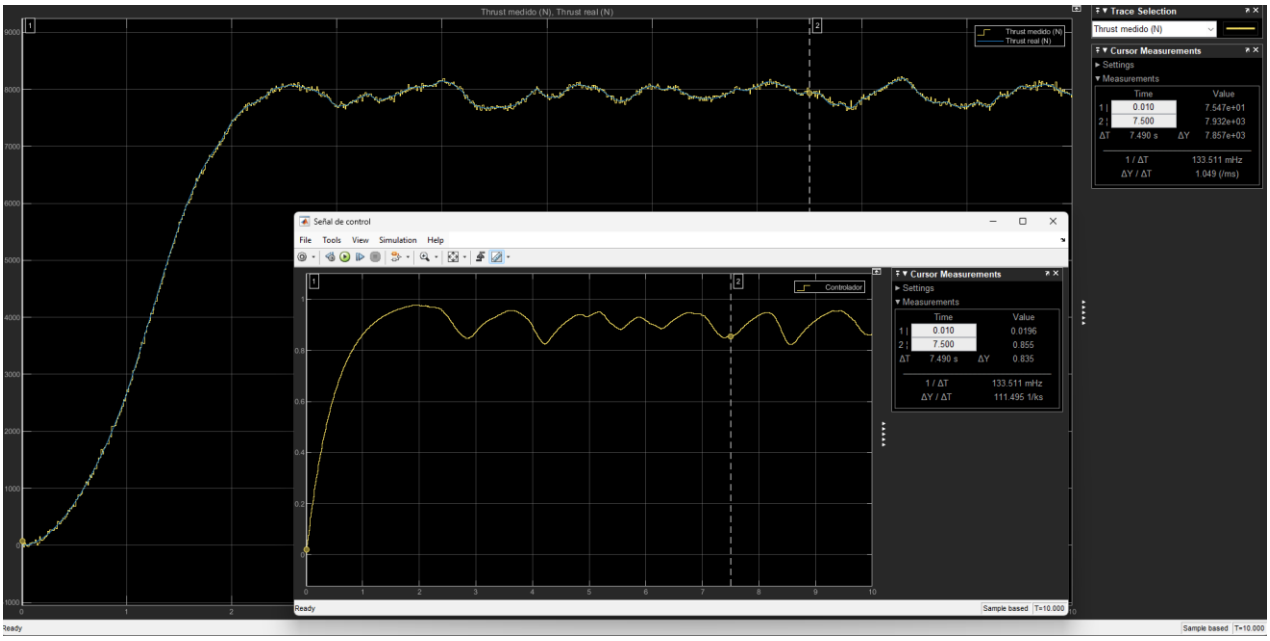

Figura 9. Obtención de resultados en Simulink

```
PS C:\Users\joleh\Desktop\MII\SEGUNDO\2Cuatrimestre\Pr1_Jolehisy_Acevedo_Medina\Controlador_grt_rtw> .\main.exe
--- Validacion SIL del Controlador (t = 0.01s) ---
Consigna de Empuje : 8000 N
Empuje Medido      : 75.47 N
--> Comando de Valvula (Throttle): 0.01961
```

Figura 10. Resultados del Código C++

5. Empaquetado del Modelo Protegido (.slxp)

Para finalizar el proyecto, se ha encapsulado el sistema completo (Controlador + Planta) en un Modelo Protegido de Simulink. Este es el estándar profesional para entregar software sin exponer el diseño interno.

5.1. Pasos realizados

1. **Configuración Atómica:** Se configuró el subsistema como una unidad atómica
2. **Conversión:** Se transformó el bloque en una "Model Reference" para permitir su aislamiento.
3. **Protección:** Se generó el archivo con extensión **.slxp**, habilitando los permisos de simulación y generación de código.

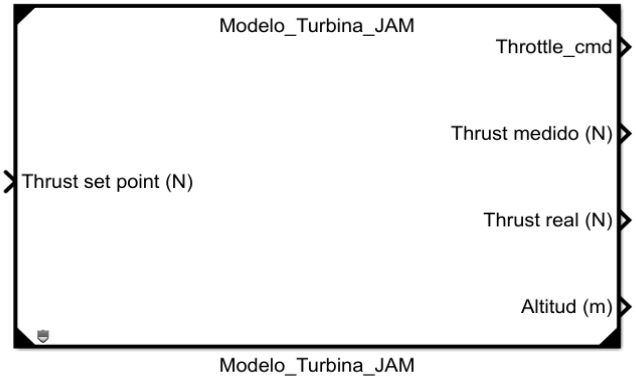

Figura 11. Modelo Protegido

El uso del formato .slxp asegura que el modelo funcione como una **"Caja Negra"**. El usuario puede simular el motor y generar código C++ sin riesgo de modificar por error las ganancias del PID o los parámetros de la turbina, garantizando la integridad de la validación realizada.

6. Conclusión General del Proyecto

El desarrollo de este Gemelo Digital ha permitido cubrir el ciclo completo de diseño de un sistema de control de una turbina, desde el modelado teórico hasta la generación de código embebido:

- **Modelado de la Planta:** Se ha creado una simulación no lineal de la turbina capaz de responder dinámicamente a las órdenes de empuje.
- **Control de Precisión:** El diseño del PID ha logrado estabilizar el sistema en el valor objetivo de **8000 N**, manteniendo la eficiencia del motor.
- **Validación de Código (SIL):** La exportación a C++ ha sido un éxito, obteniendo en el ejecutable externo una precisión idéntica a la de Simulink (**0.01961**), lo que garantiza que el software es apto para su implementación en hardware real.

El uso de **Modelos Protegidos (.slxp)** y la validación en lazo abierto demuestran un flujo de trabajo profesional, donde la seguridad y la integridad del código son tan importantes como el rendimiento del motor. Este proyecto sienta las bases para futuras pruebas de integración en tiempo real (**HIL**).